PROJECT KNOWLEDGE BASE

modern-software-dev-assignments

Build map, operations, backup, and recovery guide

Generated additively from non-sensitive project structure and existing documentation. No project launcher was executed.

Generated	2026-07-26
Purpose	Portable project orientation and recovery
Privacy	Secrets, databases, and runtime data excluded

Project knowledge, build map, and recovery guide

Generated: 2026-07-26

Project path: `D:\OneDrive\Documents\GitHub\modern-software-dev-assignments`

Project type: Web application or browser dashboard

Documentation method: Automated safe inventory plus existing project documentation

Privacy boundary: Credential-like files, tokens, databases, runtime data, caches, and generated outputs were not read

This is an additive knowledge snapshot. It does not certify correctness, security, production readiness, financial performance, or regulatory compliance. Verify it against the current source before material changes.

1. What this project is

Assignments for CS146S: The Modern Software Developer

Purpose description derived from `README.md`.

Technical shape

- **Detected stack:** Python, JavaScript, HTML, CSS, Python package/environment
- **Analyzed non-sensitive files:** 120
- **Analyzed size:** 0.28 MB
- **Git repository:** Yes
- **Current branch:** master
- **Latest commit:** ca2df55b78d6194612b65ae3fbfaa55a4678a683\|2025-11-10\|edit assignment.md
- **Working-tree summary at generation:** Clean

2. How it is made

The project is assembled from the source modules, launchers, configuration files, dependencies, and assets inventoried below. Python modules were parsed for top-level class and function names without importing or executing them. Other source types were inventoried by path and language. No live service, broker, database, or credential connection was attempted.

Dependencies

None detected automatically.

Package-defined commands

- None detected automatically.

Launchers

- None detected automatically.

Likely entry points

- `week2/app/main.py`
- `week4/backend/app/main.py`
- `week5/backend/app/main.py`
- `week6/backend/app/main.py`
- `week7/backend/app/main.py`

3. Architecture and module map

Python module structure

Module	Lines	Top-level classes/functions
week1/tool_calling.py	168	_annotation_to_str, _list_function_return_types, output_every_func_return_type, add, greet, resolve_path, extract_tool_call
week1/reflexion.py	153	extract_code_block, load_function_from_code, evaluate_function, generate_initial_function, your_build_reflexion_context, apply_reflexion, run_reflexion_flow
week6/backend/app/routers/notes.py	132	list_notes, create_note, patch_note, get_note, unsafe_search, debug_hash_md5, debug_eval
week1/rag.py	123	load_corpus_from_files, YOUR_CONTEXT_PROVIDER, make_user_prompt, extract_code_block, test_your_prompt
week2/app/db.py	116	ensure_data_directory_exists, get_connection, init_db, insert_note, list_notes, get_note, insert_action_items
week1/self_consistency_prompting.py	86	extract_final_answer, test_your_prompt
week6/backend/app/routers/action_items.py	72	list_items, create_item, complete_item, patch_item
week7/backend/app/routers/action_items.py	72	list_items, create_item, complete_item, patch_item
week2/app/services/extract.py	89	_is_action_line, extract_action_items, _looks_imperative
week7/backend/app/routers/notes.py	68	list_notes, create_note, patch_note, get_note
week1/chain_of_thought.py	72	extract_final_answer, test_your_prompt
week4/backend/app/routers/notes.py	47	list_notes, create_note, search_notes, get_note
week5/backend/app/routers/notes.py	47	list_notes, create_note, search_notes, get_note
week6/backend/app/db.py	58	get_db, get_session, apply_seed_if_needed
week7/backend/app/db.py	58	get_db, get_session, apply_seed_if_needed
week4/backend/app/db.py	56	get_db, get_session, apply_seed_if_needed
week5/backend/app/db.py	56	get_db, get_session, apply_seed_if_needed
week2/app/routers/action_items.py	50	extract, list_all, mark_done

File-type profile

Extension	Count
.py	74
.md	21
[none]	5
.html	5
.yaml	4
.js	4
.css	4
.lock	1
.toml	1
.txt	1

This structural map is intended for orientation. Detailed runtime data flow should be confirmed from the entry points, launchers, and README before modifying behavior.

4. Important project files

Path	Type	Bytes
poetry.lock	.lock	133,613
pyproject.toml	.toml	941
README.md	.md	850
.gitignore	[none]	477
week4/assignment.md	.md	7,240
week1/tool_calling.py	.py	5,433
week5/docs/TASKS.md	.md	5,288
week1/reflexion.py	.py	5,159
week5/assignment.md	.md	4,737
week8/assignment.md	.md	4,497
week2/assignment.md	.md	4,272
week6/backend/app/routers/notes.py	.py	4,189
week1/rag.py	.py	4,016
week3/assignment.md	.md	3,765
week2/app/db.py	.py	3,628
week2/frontend/index.html	.html	3,082
week6/frontend/app.js	.js	3,021
week7/frontend/app.js	.js	3,006
week7/assignment.md	.md	2,997
week1/self_consistency_prompting.py	.py	2,871
week6/assignment.md	.md	2,542
week6/backend/app/routers/action_items.py	.py	2,536
week7/backend/app/routers/action_items.py	.py	2,536
week2/app/services/extract.py	.py	2,471

Large generated, dependency, data, cache, and knowledge-output directories are excluded from this table.

5. Configuration map

Only configuration key names are recorded; values are intentionally omitted.

- No readable non-sensitive JSON configuration keys were detected.

Before using this project on another computer, review path settings, service endpoints, environment variables, account-specific values, and OS-specific launchers. Never copy a credential into documentation or Git merely to make a restored project start.

6. Run and build orientation

1. Read the project README and any adjacent documentation.
2. Review the listed launchers and package-defined commands.
3. Install dependencies in an isolated environment appropriate to the stack.
4. Restore required non-Git assets and secrets through secure channels.
5. Run tests before launching production or live-connected behavior.
6. Start in an offline, dry-run, development, or otherwise safe mode when one

is available.

No launcher was executed while generating this document.

7. Tests and verification

Detected test-related paths:

- `week2/tests/test_extract.py`
- `week4/backend/tests/conftest.py`
- `week4/backend/tests/test_action_items.py`
- `week4/backend/tests/test_extract.py`
- `week4/backend/tests/test_notes.py`
- `week5/backend/tests/conftest.py`
- `week5/backend/tests/test_action_items.py`
- `week5/backend/tests/test_extract.py`
- `week5/backend/tests/test_notes.py`
- `week6/backend/tests/conftest.py`
- `week6/backend/tests/test_action_items.py`
- `week6/backend/tests/test_extract.py`
- `week6/backend/tests/test_notes.py`
- `week7/backend/tests/conftest.py`
- `week7/backend/tests/test_action_items.py`
- `week7/backend/tests/test_extract.py`
- `week7/backend/tests/test_notes.py`

The absence of detected tests does not prove there are none, and the presence of tests does not prove they pass. This batch documentation process did not run project launchers, tests, training jobs, network calls, or live integrations.

Recommended verification sequence:

1. Confirm dependency installation.
2. Run the repository's documented test command.
3. Validate configuration without exposing values.
4. Compare behavior with the README and expected outputs.
5. Review version-control changes before deployment or live use.

8. Data, privacy, and backup boundaries

The scanner intentionally skipped 0 credential-like or database-like file paths, plus standard excluded directories such as `data/`, `.git/`, `node_modules/`, virtual environments, caches, generated outputs, and temporary files.

Git alone may therefore be insufficient. Keep separate protected backups for:

- Databases and runtime history
- Model weights, large datasets, and generated research artifacts
- Environment-variable values and credentials
- External scripts or files referenced by absolute paths
- OS keychain or Credential Manager recovery information

- Files intentionally excluded by `.gitignore`

Do not place secret values in `KNOWLEDGE.md`, PDFs, source control, or a public cloud location.

9. New-laptop recovery

1. Restore or clone the complete project folder.
2. Open the folder in Codex and read `KNOWLEDGE.md` plus the detailed guide.
3. Install the detected language runtimes and dependencies.
4. Restore private data and large artifacts separately.
5. Recreate credentials securely.
6. Update absolute paths and machine-specific configuration.
7. Run tests and non-live validation.
8. Verify outputs before enabling external or consequential actions.

Suggested Codex opening prompt

> Read KNOWLEDGE.md, the detailed project knowledge guide, the README, and the > current Git status. Do not expose secrets or change project behavior. First > explain the architecture, entry points, dependencies, data boundaries, and > safest verification command. Report any mismatch between documentation and > source.

10. Maintenance rules

- Keep Markdown as the editable source and PDF as a portable snapshot.
- Regenerate after material architecture, dependency, launcher, configuration,

data-model, or deployment changes.

- Add the exact Git commit hash after a stable baseline is committed.
- Do not overwrite project-authored documentation with generated text.
- Review automated inferences before treating them as authoritative.
- Keep credentials and private runtime data outside the knowledge pack.

11. Source basis

This guide used:

- Existing README text when available
- Non-sensitive source and launcher filenames
- Python abstract syntax trees, without importing the modules
- Dependency manifests
- Non-sensitive JSON key names, without values
- Local Git metadata, without remote URLs

Generated knowledge files and PDFs were excluded from the analysis to avoid self-referential inventory growth.